MODULE 1 · LECTURE 21

Search Strategies in AI: Breadth-first Search and Depth-first Search

How uninformed search explores a state space, how strategies are compared on completeness, optimality, time and memory, and how breadth-first, depth-first, depth-limited, iterative-deepening and uniform-cost search behave — traced by hand and run on a field-robot problem.

Learning objectives

After this lecture you should be able to:

1. Explain search trees, nodes, the frontier and the explored set.
2. Evaluate search strategies by completeness, optimality, time and space complexity.
3. Trace breadth-first and depth-first search by hand and state their properties.
4. Explain depth-limited, iterative-deepening, uniform-cost and bidirectional search.
5. Compare uninformed search strategies and estimate the memory needed by breadth-first search.
6. Apply search to agricultural problems such as robot path finding.

Scope: Lecture 20 introduced state spaces, production systems and control strategies, and solved the water-jug problem. This lecture studies the main **uninformed (blind)** strategies in detail. **Heuristic** (informed) search, which uses an estimate of the distance to the goal, follows in Lecture 22.

1. Searching a state space

A search algorithm builds a **search tree**: the root is the initial state, and the children of a node are the states reached by applying each operator. **Expanding** a node means generating its children. The algorithm keeps a **frontier** (open list) of generated but unexpanded nodes and an **explored set** (closed list) of states already expanded, so that it does not repeat them (Russell & Norvig, 2020). A search strategy is simply a rule for choosing which frontier node to expand next.

1. Put the initial state on the frontier.
2. If the frontier is empty, report failure.
3. Remove a node from the frontier according to the strategy.
4. If it is a goal, return the path to it.
5. Otherwise add it to the explored set, generate its children, put new ones on the frontier, and go to step 2.

How strategies are compared

Criterion	Question
Completeness	Is it guaranteed to find a solution when one exists?
Optimality	Does it find the best (lowest-cost) solution?
Time complexity	How many nodes does it generate or expand?
Space complexity	How many nodes must it keep in memory?

Complexity is expressed with **b**, the branching factor (the number of children per node), **d**, the depth of the shallowest solution, and **m**, the maximum depth of the tree.

2. Breadth-first search (BFS)

Breadth-first search expands the shallowest unexpanded node first: all nodes at depth 1, then all at depth 2, and so on. Its frontier is a **first-in, first-out (FIFO) queue** — new nodes join the back and the oldest is taken from the front.

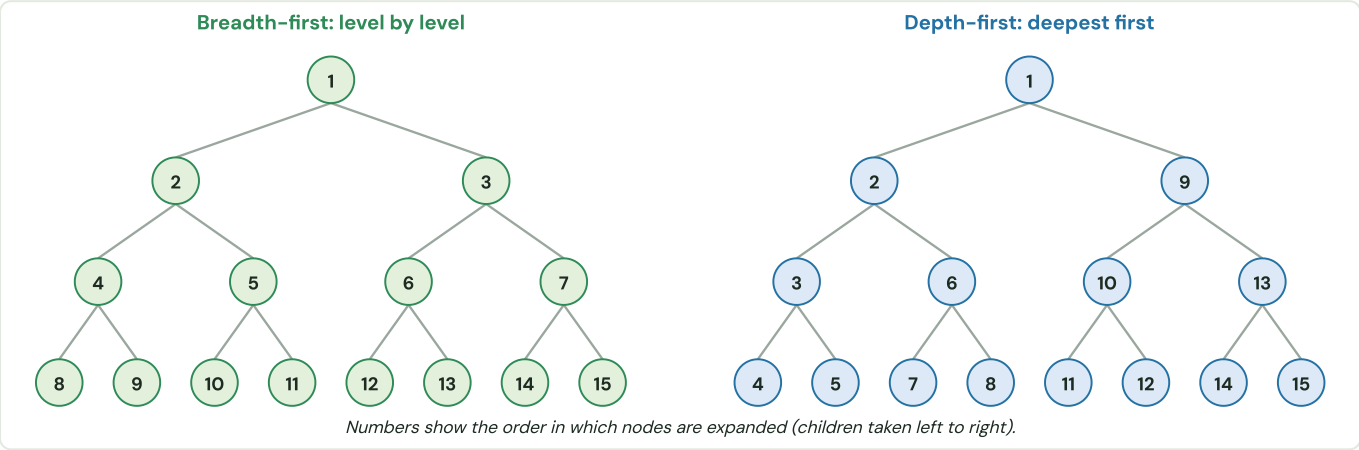


Figure 1. Order of node expansion in breadth-first and depth-first search on the same tree. Original diagram.

Worked example 1 — a farm road network

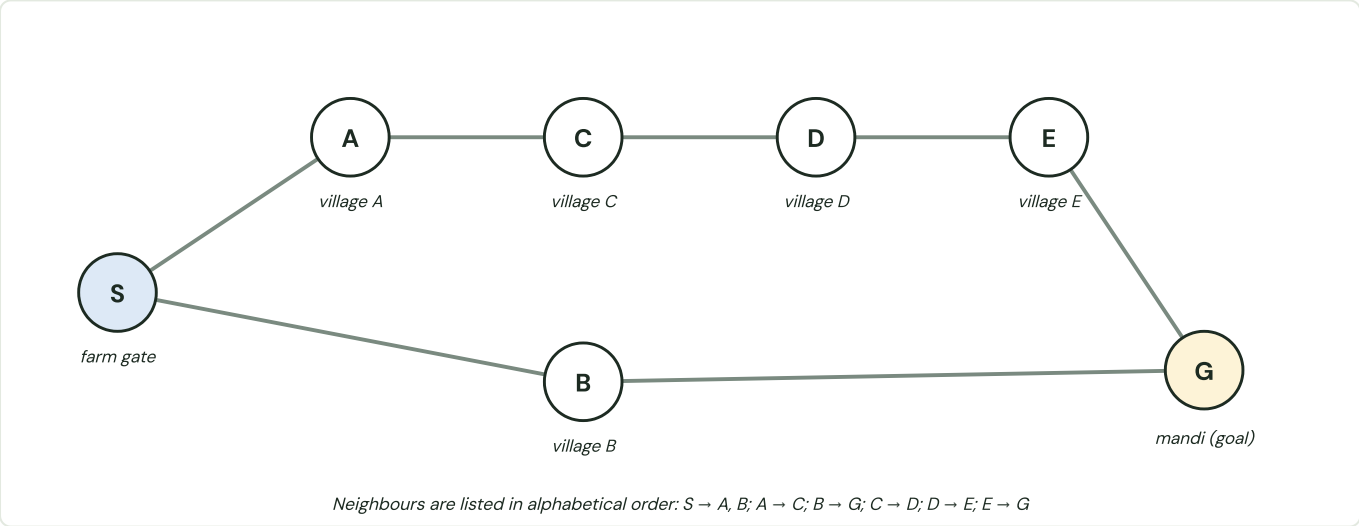


Figure 2. A farm road network: find a route from the farm gate S to the mandi G. Original diagram.

Step	Node removed (BFS, queue)	Children added	Frontier after the step
1	S	A, B	A, B
2	A	C	B, C
3	B	G	C, G
4	C	D	G, D
5	G — goal	—	Path S → B → G (2 roads)

Properties of BFS

- **Complete** whenever the branching factor is finite — it will eventually reach every depth.

- **Optimal** when every step costs the same, because it finds the shallowest goal first.
- **Time and space O(b^d)** — it must keep a whole level of the tree in memory, which is its main weakness (Russell & Norvig, 2020).

> **Worked example 2 — how quickly BFS memory grows** (assumptions: b = 10, 1 million nodes generated per second, 1 kilobyte per node)
> Nodes up to depth d = $1 + 10 + 10^2 + \ldots + 10^d$.
> d = 2: 111 nodes — 0.1 ms, 111 KB · d = 4: 11,111 nodes — 11 ms, 11 MB · d = 6: 1,111,111 nodes — 1.1 s, 1.1 GB
> d = 8: 111 million nodes — 111 s, 111 GB · d = 10: 11.1 billion nodes — 3.1 hours, 11.1 TB · d = 12: 1.1 trillion nodes — 12.9 days, 1.1 PB
> Memory, not time, runs out first: breadth-first search becomes impractical on large problems.

3. Depth-first search (DFS)

Depth-first search always expands the deepest node on the frontier, going down one branch until it reaches a dead end and then backing up to the most recent unexplored alternative. Its frontier is a **last-in, first-out (LIFO) stack**. Depth-first search is also the basis of many efficient graph algorithms (Tarjan, 1972).

Step	Node removed (DFS, stack)	Children pushed	Stack after the step (top on the right)
1	S	B, A (so that A is on top)	B, A
2	A	C	B, C
3	C	D	B, D
4	D	E	B, E
5	E	G	B, G
6	G — goal	—	Path S → A → C → D → E → G (5 roads)

On the same network, DFS found a route of 5 roads while BFS found one of 2: depth-first search returns the first path it stumbles on, not the shortest.

Properties of DFS

- **Not complete** in general: it can follow an infinite path, or loop forever unless repeated states are checked. In a finite space with repeated-state checking it does find a solution.
- **Not optimal**: the first solution found may be long, as above.
- **Time O(b^m)**, which is poor if the maximum depth m is much larger than d.
- **Space O(b·m)** — only the current path and the unexplored siblings along it are stored. This small memory need is its great advantage.

4. Variants of uninformed search

Strategy	Idea
Depth-limited search (DLS)	Depth-first search that does not go below a fixed depth limit l — avoids infinite paths, but fails if the shallowest goal is deeper than l
Iterative deepening search (IDS)	Runs depth-limited search with limits 0, 1, 2, … until a goal is found. It combines the small memory of DFS with the completeness and shortest-path guarantee of BFS (Korf, 1985)
Uniform-cost search (UCS)	Expands the node with the lowest path cost so far — finds the cheapest path when steps have different costs, as in Dijkstra's (1959) shortest-path algorithm
Bidirectional search	Searches forward from the start and backward from the goal at the same time and stops when the two meet — roughly O(b^(d/2)) instead of O(b^d)

Worked example 3 — iterative deepening on the farm road network
Limit 0: expands S only — no goal.
Limit 1: S, A, B — no goal (G is at depth 2).
Limit 2: S, A, C (stop: depth limit), B, G — goal found: S → B → G, the same shortest route as BFS, while storing only one path at a time.
The repeated work at small depths costs little, because most nodes of a tree lie at its deepest level.

Comparison

Strategy	Complete?	Optimal?	Time	Space
Breadth-first	Yes (finite b)	Yes, if step costs are equal	O(b^d)	O(b^d)
Depth-first	No (yes in finite spaces with repeated-state checks)	No	O(b^m)	O(b·m)
Depth-limited	No (only if l ≥ d)	No	O(b^l)	O(b·l)
Iterative deepening	Yes (finite b)	Yes, if step costs are equal	O(b^d)	O(b·d)
Uniform-cost	Yes, if every step costs more than zero	Yes	Exponential in cost of best solution	Same as time
Bidirectional (BFS both ways)	Yes (finite b)	Yes, if step costs are equal	O(b^(d/2))	O(b^(d/2))

Complexities follow Russell and Norvig (2020); breadth-first and depth-first graph traversal are also treated in Cormen et al. (2022).

5. Worked example: a field robot

A small robot must travel from its start cell S to a sampling point G in a field mapped as a 6 × 6 grid, moving up, down, left or right, and avoiding cells with trees, wells or boulders.

Breadth-first and depth-first search for a field robot in Python (Lecture 8)

```
from collections import deque

FIELD = ["S . . . . .",      # S = robot start, G = goal, # = obstacle (tree, well, boulder)
         ". . . . . .",
         ". . # # . .",
         ". . # . . .",
         ". . . . # .",
         ". . . . . G"]
grid = [row.split() for row in FIELD]
ROWS, COLS = len(grid), len(grid[0])
start = (0, 0); goal = (ROWS - 1, COLS - 1)

def neighbours(cell):                       # up, down, left, right moves inside the field
    r, c = cell
    for dr, dc in ((-1, 0), (1, 0), (0, -1), (0, 1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < ROWS and 0 <= nc < COLS and grid[nr][nc] != "#":
            yield (nr, nc)

def search(kind):
    frontier = deque([(start, [start])])
    seen, expanded = {start}, 0
    while frontier:
        cell, path = frontier.popleft() if kind == "BFS" else frontier.pop()
        expanded += 1
        if cell == goal:
            return path, expanded
        for nxt in neighbours(cell):
            if nxt not in seen:
                seen.add(nxt)
                frontier.append((nxt, path + [nxt]))

for kind in ("BFS", "DFS"):
    path, expanded = search(kind)
    print(f"{kind}: path of {len(path) - 1} moves, {expanded} cells expanded")
    show = [row[:] for row in grid]
    for r, c in path[1:-1]:
        show[r][c] = "*"
    print("\n".join("   " + " ".join(row) for row in show))

Output:
BFS: path of 10 moves, 32 cells expanded
   S . . . . .
   * . . . . .
   * . # # . .
   * . # . . .
   * . . . # .
   * * * * * G
DFS: path of 14 moves, 25 cells expanded
   S * * * * *
   . . . . . *
   . . # # * *
   . . # * * .
   . . . * # .
   . . . * * G
```

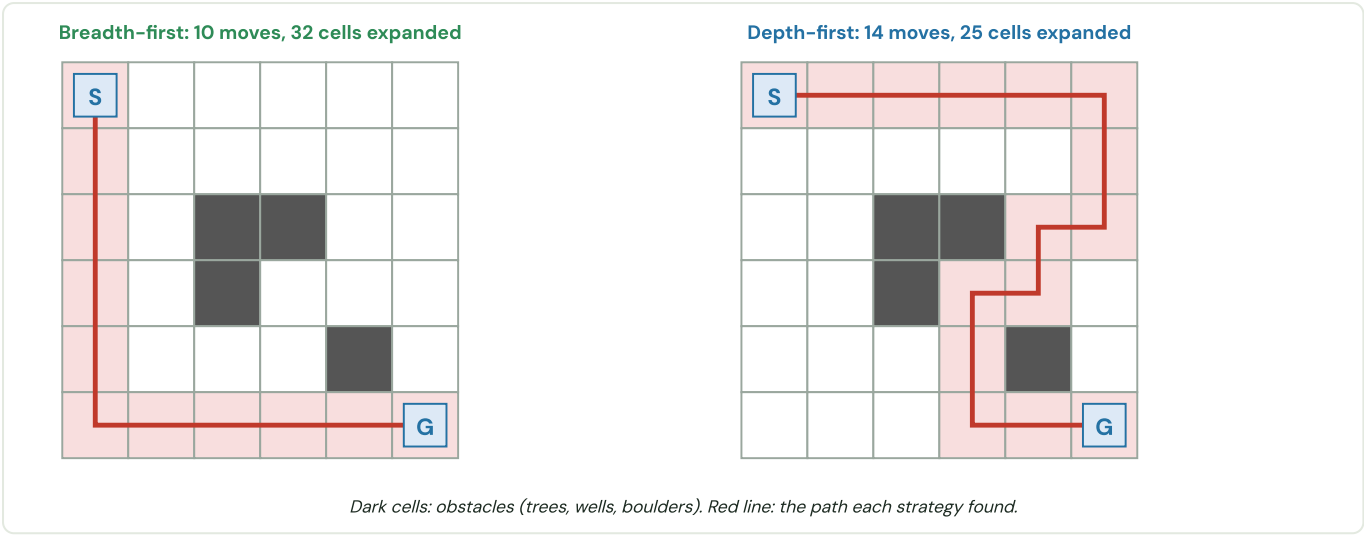


Figure 3. Paths found by breadth-first and depth-first search on the same field. Original diagram generated from the program's rules.

The program was run and the output shown is its actual output. BFS found the shortest route (10 moves) but expanded more cells (32); DFS expanded fewer (25) but took a longer route (14 moves). Heuristic search (Lecture 22) aims to get the short route while expanding fewer cells.

6. Uses in agriculture

Problem	How search is used
Robot and machine navigation	Finding routes for field robots, sprayers and harvesters around obstacles on a grid or road map (Lectures 24 and 27)
Counting fields or patches in a classified image	BFS or DFS "flood fill" visits all connected pixels of one class, so each connected patch — a field, a pond, a weed patch — is found and measured (Lecture 15)
Diagnosis	Systematically exploring possible causes of a symptom, as in expert systems (Lecture 17)
Planning and scheduling	Exploring sequences of operations, such as the order in which to irrigate zones with limited water (Lecture 12)
Puzzles and teaching problems	Water-jug and river-crossing problems, used to compare strategies (Lecture 20)

7. Key takeaways

- Search builds a tree from the initial state; the frontier holds generated nodes, the explored set prevents repeats.
- Strategies are judged on completeness, optimality, time and space, using b, d and m.
- Breadth-first search uses a FIFO queue; it is complete and finds the shortest path for equal step costs, but needs O(b^d) memory.
- Depth-first search uses a LIFO stack; it needs only O(b·m) memory but is neither complete in general nor optimal.
- Depth-limited search caps depth; iterative deepening repeats it with growing limits, combining DFS memory with BFS guarantees.
- Uniform-cost search finds the cheapest path when costs differ; bidirectional search meets in the middle.

- In agriculture, these strategies support robot navigation, image analysis, diagnosis and scheduling.

8. Examination questions

Short-answer questions (2 marks)

Q1. What is the frontier in a search algorithm?

The set of generated but not yet expanded nodes (the open list).

Q2. Name the four criteria for evaluating search strategies.

Completeness, optimality, time complexity and space complexity.

Q3. What data structure does BFS use for its frontier?

A first-in, first-out (FIFO) queue.

Q4. What data structure does DFS use for its frontier?

A last-in, first-out (LIFO) stack.

Q5. State the time and space complexity of BFS.

O(b^d) for both.

Q6. Why is DFS memory-efficient?

It stores only the current path and the unexplored siblings along it — O(b·m).

Q7. When is BFS optimal?

When all step costs are equal.

Q8. What is depth-limited search?

Depth-first search that does not go below a fixed depth limit.

Q9. What is iterative deepening search?

Repeated depth-limited search with limits 0, 1, 2, … until a goal is found.

Q10. Which search finds the cheapest path when step costs differ?

Uniform-cost search.

Long-answer questions (5–10 marks) — answer outlines

Q1. Explain the general search algorithm and the criteria used to evaluate search strategies.

Search tree, frontier, explored set, steps; completeness, optimality, time, space; b, d, m.

Q2. Explain breadth-first search with an example and state its properties.

FIFO queue; Figure 1; worked example 1 trace; properties; worked example 2 memory table.

Q3. Explain depth-first search with an example and state its properties.

LIFO stack; Figure 1; DFS trace on the farm network; properties; dangers of infinite paths.

Q4. Explain depth-limited and iterative deepening search and why iterative deepening is preferred.

Definitions; worked example 3; complexity table.

Q5. Compare BFS, DFS, IDS and uniform-cost search.

Comparison table; when to use each.

Q6. Apply BFS and DFS to a robot path-finding problem and interpret the results.

Program, output and Figure 3; shortest path vs expansions; agricultural uses.

9. Glossary

Term	Meaning
Branching factor (b)	Number of children of a node
Completeness	Guarantee of finding a solution if one exists
Depth (d)	Depth of the shallowest goal
Depth-limited search	DFS with a maximum depth
Expanding a node	Generating its children
Explored set	States already expanded (closed list)
FIFO queue	First in, first out — used by BFS
Flood fill	Visiting all connected cells of the same class
Frontier	Generated but unexpanded nodes (open list)
Iterative deepening	Repeated depth-limited search with increasing limits
LIFO stack	Last in, first out — used by DFS
Maximum depth (m)	Deepest level of the search tree
Optimality	Guarantee of finding the lowest-cost solution
Search tree	Tree of states generated from the initial state
Uniform-cost search	Expands the node with the lowest path cost

References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press. ISBN 978-0-262-04630-5

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik, 1*(1), 269–271. https://doi.org/10.1007/BF01386390

Korf, R. E. (1985). Depth-first iterative-deepening: An optimal admissible tree search. *Artificial Intelligence, 27*(1), 97–109. https://doi.org/10.1016/0004-3702(85)90084-0

Russell, S., & Norvig, P. (2020). *Artificial intelligence: A modern approach* (4th ed.). Pearson. ISBN 978-0-13-461099-3

Tarjan, R. (1972). Depth-first search and linear graph algorithms. *SIAM Journal on Computing, 1*(2), 146–160. https://doi.org/10.1137/0201010

All diagrams and programs are original. The road network, field grid and memory assumptions are illustrative; every trace and result shown was checked by running the algorithms.

Practice MCQs

Q1. The frontier (open list) contains

(a) all states in the problem
(b) generated but not yet expanded nodes
(c) only goal states
(d) expanded nodes

Q2. Breadth-first search uses which data structure for its frontier?

(a) Stack (LIFO)
(b) Queue (FIFO)
(c) Priority queue by heuristic
(d) Hash table only

Q3. Depth-first search uses which data structure for its frontier?

(a) Stack (LIFO) (b) Queue (FIFO)
(c) Sorted list (d) Tree only

Q4. The space complexity of breadth-first search is

(a) O(b·m) (b) O(b^d)
(c) O(d) (d) O(1)

Q5. The space complexity of depth-first search is

(a) O(b·m) (b) O(b^d)
(c) O(b^m) (d) O(d²)

Q6. Breadth-first search is optimal when

(a) the tree is infinite (b) all step costs are equal
(c) a heuristic is used (d) the goal is deep

Q7. Which search may follow an infinite path and never return?

(a) Breadth-first search (b) Uniform-cost search
(c) Depth-first search (d) Iterative deepening

Q8. Iterative deepening search combines

(a) the memory of BFS and the speed of DFS (b) the memory of DFS and the guarantees of BFS
(c) heuristics and costs (d) two goals

Q9. The search that expands the node with the lowest path cost is

(a) depth-first (b) breadth-first
(c) uniform-cost (d) depth-limited

Q10. With b = 10, how many nodes does BFS generate up to depth 4?

(a) 40 (b) 1,000
(c) 11,111 (d) 1,000,000

Q11. In the farm road example, BFS found S → B → G and DFS found

(a) S → B → G (b) S → A → C → D → E → G
(c) no path (d) S → G

Q12. "Flood fill" for counting fields in a classified image is an application of

(a) BFS or DFS (b) linear programming
(c) regression (d) certainty factors

Answers

Q1 — (b) generated but not yet expanded nodes
Q2 — (b) Queue (FIFO)
Q3 — (a) Stack (LIFO)
Q4 — (b) O(b^d)
Q5 — (a) O(b·m)
Q6 — (b) all step costs are equal
Q7 — (c) Depth-first search
Q8 — (b) the memory of DFS and the guarantees of BFS
Q9 — (c) uniform-cost
Q10 — (c) 11,111
Q11 — (b) S → A → C → D → E → G
Q12 — (a) BFS or DFS